

- 01 Step 0: 환경 진단
- 02 Step 1: prd-reviewer sub-agent 작성
- 03 Step 2: git-committer sub-agent 작성
- 04 Step 3: hook 작성 (PostToolUse + Stop)
- 05 Step 4: 화면 1 나머지 Task 빌드
- 06 Step 5: 화면 2 빌드 (패턴 복사)
- 07 Step N: 정리와 비용 비교
- 08 과제 + Tips

— STEP 0

환경 진단

자료 5 산출물 점검 + 토큰 베이스라인

- 01 0-1. 환경 확인 명령 (node·claude·git·gh + planning_project)
- 02 0-2. 자료 5 산출물 5개 점검
- 03 0-3. 점검 명령 (docs/tasks.md, gh issue, ls)
- 04 0-4. 토큰 베이스라인 (/cost 메모)

— 활동 0-1 · 산출물: 4 도구 확인 + PLANNING_PROJECT 진입

환경 확인 명령 — node·claude·git·gh

통과 조건: node 18.0 이상, claude·git·gh 모두 동작.

MACOS / LINUX

```
node --version
claude --version
git --version
gh --version
cd planning_project
```

WINDOWS (POWERSHELL)

```
node --version
claude --version
git --version
gh --version
Set-Location planning_project
```


자료 5 산출물 5개 점검

비어 있는 항목이 있으면 자료 5의 해당 Step으로 돌아가 보강 후 본 단계 진입.

01 docs/tasks.md 화면 단위 작업 목록 있는가	02 GitHub Issue 10개 screen-1 4개, screen-2 6개 등록됐는가	03 첫 PR (Issue #1) 병합된 상태 병합됐는가	04 InputForm.tsx 입력 폼 컴포넌트 동작하는가	05 빌드 명령 메모 손에 있는 상태 손에 있는가
--	--	---	--	---

— 활동 0-3 · 산출물 확인 명령

점검 명령 — 3가지

자료 5 Step으로 돌아가 보강 필요 여부 판단하기.

BASH

planning_project 폴더에서 실행

```
cat docs/tasks.md | head -30
gh issue list --label task
ls src/components/
```

— 활동 0-4 · 토큰 베이스라인

현재 토큰 상태 확인

Step 1~5 완료 후 최종 비용을 비교하기 위해 지금 기준값을 메모해두기.

BASH

Claude CLI 명령으로 현재 토큰 사용량 확인

/cost

— STEP 0 · 검증 / 성공 기준

다음 Step 진입 가능한가

3개 항목 모두 통과 시 Step 1 (prd-reviewer sub-agent 작성)로 진입.

01 ☒ planning_project 폴더에서 작업 중인가

02 ☒ 자료 5 산출물 5개가 모두 있는가

03 ☒ /cost 베이스라인을 메모했는가

— STEP 1

prd-reviewer sub-agent 작성

본 단계 목표는 6원칙을 따른 prd-reviewer 작성과 호출 테스트

- 01 1-1. .claude/agents 디렉토리 생성
- 02 1-2. prd-reviewer.md 작성 (frontmatter 4필드 + 역할 정의)
- 03 1-3. 호출 테스트 및 보고서 생성 (3카테고리 형식)

활동 1-1 · 산출물: .CLAUDE/AGENTS 디렉토리

1-1. 디렉토리 생성

planning_project 폴더 안에 숨겨진 .claude/agents 디렉토리를 생성합니다.

MACOS / LINUX

mkdir -p .claude/agents

WINDOWS (POWERSHELL)

New-Item -ItemType Directory -Path .claude/agents -Force

1-2. prd-reviewer.md 작성

모든 sub-agent는 이 구조를 따른다. frontmatter 4필드 + 본문 역할/입력/절차/출력/주의.

파일 경로: `.claude/agents/prd-reviewer.md`

name: prd-reviewer

description: 코드 작성/수정 직후 호출.

docs/prd.md를 읽고 현재 코드와 비교해 일치성을 보고합니다.

tools: Read, Grep, Glob

model: sonnet

PRD 일치성 검토 전문가의 역할을 수행합니다.

****입력****: 방금 수정된 파일의 경로

****작업 절차****:

- docs/prd.md를 Read로 읽고 핵심 기능 3개와 제약사항 메모
- 입력 파일을 Read로 읽기
- 코드 기능을 PRD 핵심 기능에 매핑
- rules/coding-style.md와 rules/architecture.md 점검

****출력 형식**** (반드시 3카테고리로):

- 일치: PRD 핵심 기능과 매핑
- 불일치: 임의 기능 추가 또는 모호한 표현
- 검토 외: rules/ 가이드 점검

****주의****:

- 수정 권한 없음 (tools에 Write/Edit 없음)
- 추측하지 말고 PRD 그대로 비교
- 보고서는 한국어

1-3. 호출 테스트 및 보고서

sub-agent를 호출하고 3카테고리 형식의 보고서를 받습니다.

호출 명령

Claude Code 세션:

@prd-reviewer src/components/InputForm.tsx 파일을 검토해줘.

시드 6번 보고서 예시 (INPUTFORM.TSX)

[일치]

- 상품 사진 업로드 영역
→ PRD 핵심 기능 1(입력)에 매핑
- 키워드 입력 5칸
→ PRD 핵심 기능 1에 명확히 매핑

[불일치]

- 발견된 불일치 없음

[검토 외]

- rules/coding-style.md의 PascalCase 컴포넌트명 규칙 준수
- PR 1-2개 파일 원칙에 부합

STEP 1 · 검증 / 성공 기준

다음 Step 진입 가능한가

4개 항목 모두 통과 시 Step 2 (git-committer sub-agent 작성)로 진입.

- 01

☒

.claude/agents/prd-reviewer.md 파일이 작성됐는가
- 02

☒

frontmatter에 mcpServers 필드가 없는가
- 03

☒

@prd-reviewer 호출에 sub-agent가 응답하는가
- 04

☒

보고서가 3카테고리([일치]/[불일치]/[검토 외])로 출력됐는가

— STEP 2

git-committer sub-agent 작성

본 단계 목표는 자동 커밋 sub-agent 작성과 호출 테스트

- 01 2-1. git-committer.md 작성 (frontmatter 4필드 + 역할 정의)
- 02 2-2. 호출 테스트 명령과 보고서 예시 (3단 구조)
- 03 2-3. Step 2 검증 기준 (5개 항목)

2-1. git-committer.md 작성

모든 sub-agent는 이 구조를 따른다. frontmatter 4필드 + 본문 역할/입력/절차/출력/주의.

파일 경로: `.claude/agents/git-committer.md`

```
---
name: git-committer
description: 메인 에이전트 응답 종료 시 호출.
  변경된 파일을 분석해 CLAUDE.md 형식대로 커밋·push.
tools: Bash, Read
model: sonnet
---

git 커밋 자동화 전문가의 역할.

**입력**: 응답 종료 신호 (변경 파일은 git status로 감지)

**작업 절차**:

1. `git status --porcelain` 실행 → 변경 파일 확인
2. 변경 없으면 즉시 종료 "변경 없음" 보고
3. CLAUDE.md를 Read로 읽고 커밋 형식 확인
4. rules/commit-convention.md를 Read로 읽고 type 목록 확인
5. `git diff --stat`으로 변경 요약
6. [type] description 형식 메시지 작성
7. git add → git commit -m "..." → git push

**출력 형식** (3단 구조):
- 변경 파일: 목록
- 커밋 메시지: 전문
- 실행 결과: push 성공 여부와 commit hash

**주의**:
- 변경 없으면 커밋 안 함
- type은 rules/commit-convention.md에 정의된 것만
- 추측 금지. git status 그대로 처리
- 보고서는 한국어
```


— 활동 2-2 · 산출물: 호출 테스트 및 보고서

2-2. 호출 테스트 및 보고서

sub-agent를 호출하고 3단 형식의 보고서를 받습니다.

호출 명령 및 준비

작은 변경 만들기:

```
echo "" >> README.md
```

Claude Code 세션:

@git-committer 변경된 파일을 커밋해줘.

시드 6번 보고서 예시 (README 변경)

[변경 파일]

- README.md (수정됨, 1줄 추가)

[커밋 메시지]

[chore] README 빈 줄 추가

[실행 결과]

- git add README.md: 성공

- git commit: 성공 (commit hash: a1b2c3d)

- git push origin main: 성공

STEP 2 · 검증 / 성공 기준

다음 Step 진입 가능한가

5개 항목 모두 통과 시 Step 3 (hook 작성)로 진입.

- 01

☒

.claude/agents/git-committer.md 파일이 작성됐는가
- 02

☒

frontmatter가 6원칙 따랐는가 (단일 책임, Bash·Read만)
- 03

☒

@git-committer 호출에 sub-agent가 응답하는가
- 04

☒

보고서가 3단 구조로 출력됐는가
- 05

☒

GitHub 저장소에 커밋이 도착했는가

hook 작성

PostToolUse + Stop

본 단계 목표는 settings.json에 두 hook 등록과 자동 트리거 확인.

산출물 위치

planning_project/.claude/settings.json (hooks 필드 추가)

학습 목표

- PostToolUse hook: 도구 실행 후 자동 검증
- Stop hook: 응답 종료 시 자동 커밋
- 두 hook 자동 트리거 확인 및 통합 테스트

두 hook 등록

.claude/settings.json

```
{
  "permissions": {
    "auto_allow": ["Read", "Grep", "Glob", "Edit"],
    "ask_before": ["Bash", "Write", "WebFetch"],
    "deny": ["BashWithSudo"]
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "agent",
            "prompt": "@prd-reviewer 방금 수정된 파일을 검토해주세요.",
            "timeout": 90
          }
        ]
      }
    ],
    "Stop": [
      {
        "hooks": [
          {
            "type": "agent",
            "prompt": "@git-committer 변경된 파일이 있으면 커밋해주세요.",
            "timeout": 60
          }
        ]
      }
    ]
  }
}
```

핵심: 기존 `mcpServers` 필드는 제거하고 `hooks` 필드 추가. PostToolUse matcher `"Edit|Write"` 는 공백 없음. Stop은 matcher 필드 없음.

3-2. PostToolUse hook 테스트

Edit 도구 후 자동 검증 흐름

- 1

메인이 Edit 도구로 README.md 수정

작은 코드 수정 명령 (파일 변경 감지)
- 2

PostToolUse hook 자동 트리거

Edit 도구 종료 후 hook 감지 (수정 확인)
- 3

prd-reviewer 자동 호출

@prd-reviewer 프롬프트 실행 (hook 정의)
- 4

보고서 메인 세션에 표시

검증 완료 보고서 출력 (호출 완료)

결과: 한 번의 수정 명령이 자동 검증까지 완성. 검증 보고서 즉시 표시.

3-3. Stop hook 테스트

응답 종료 시 자동 커밋 흐름

- 1

Stop hook 자동 트리거

Step 3-2 응답 종료 신호 감지
- 2

git-committer 자동 호출

@git-committer 프롬프트 실행 (hook 정의)
- 3

README.md 변경 자동 커밋

git status 분석 → 커밋 메시지 작성 → push
- 4

GitHub에 커밋 도착

커밋 해시 확인 (완료)

완성: 두 hook이 한 번의 명령으로 모두 동작. 작업자는 "README에 코멘트 추가" 한 줄만 입력.

Step 3 검증 기준

아래 5개 항목을 모두 확인하면 Step 3 완료

- ☐ settings.json에 hooks 필드 추가됐는가
- ☐ mcpServers 필드가 제거됐는가
- ☐ 코드 수정 후 prd-reviewer가 자동 호출됐는가
- ☐ 응답 종료 후 git-committer가 자동 호출됐는가
- ☐ GitHub에 자동 커밋이 도착했는가

진행률

0/5

— STEP 4

화면 1 나머지 Task 빌드

Issue #2, #3, #4를 자동화 흐름으로 빌드해 화면 1 동작 3요소 완성

- 01 Issue #2 브랜치 생성 (git flow 복습)
- 02 Issue #2 빌드 명령 (프롬프트 패턴)
- 03 자동화 흐름 이해 (PostToolUse + Stop 결합)
- 04 Issue #2 실행 및 검증
- 05 화면 1 완성 여부 점검

— 활동 4-1 · ISSUE #2 브랜치 생성

4-1. Issue #2 브랜치

새 feature 브랜치를 생성하고 최신 main에서 시작합니다.

```
git checkout main  
git pull origin main  
git checkout -b feat/task-1-2-image-upload
```

main 브랜치 동기화 후 새 feature 브랜치 생성. 브랜치명 패턴: feat/{task-number}-{설명}.

4-2. Issue #2 빌드 명령

자료 5 메모 구조를 유지하되, 화면 목적을 현재 Task로 변경합니다.

"Issue #2를 진행해줘.
CLAUDE.md와 rules/architecture.md,
rules/coding-style.md를 참조해.

PRD의 카피 입력 화면 안의
이미지 업로드 처리를 Next.js + Tailwind로 만들어줘.

기능은 다음 두 가지:
1. 드래그앤드롭과 클릭 두 방식으로 상품 사진 업로드
2. 업로드된 파일을 콘솔에 출력 (실제 처리는 다음 Task에서)

이 PR은 src/lib/upload.ts와
src/components/ImageUploader.tsx 두 파일만 수정해."

프롬프트 패턴: (1) 참조 파일 명시 (2) 화면 목적 (3) 기능 2가지 (4) 수정 범위 한정.

4-2. 자동화 흐름

한 명령으로 시작된 작업이 도구의 자동화 파이프라인 완주

1	작업자 명령 명령 한 줄 입력
2	메인: 파일 작성 upload.ts + ImageUploader.tsx 작성 (Write 사용)
3	PostToolUse 혹 자동 트리거 Edit\Write matcher 매칭
4	prd-reviewer 자동 호출 sub-agent 실행 (3-category 보고서)
5	응답 작성 완료 메인이 prd-reviewer 보고서 포함해 회신
6	Stop 혹 git-committer 자동화 자동 커밋 + 푸시, GitHub 도착

작업자가 한 일은 명령 한 줄과 보고서 확인. Issue #1보다 손이 절반 이상 줄어듦.

4-3. PR 생성

gh pr create로 Issue #2 자동 닫힘

```
gh pr create ₩
--title "feat: Issue #2 이미지 업로드 처리" ₩
--body "Closes #2"
```

본인 리뷰 후 병합. PR 본문에 **Closes #2** 키워드를 포함하면 PR을 병합할 때 Issue #2가 자동으로 닫힌다.

GitHub는 PR 본문의 특정 키워드(Closes, Fixes, Resolves 등)를 감지하면, PR이 병합될 때 자동으로 연결된 Issue를 닫는다. 이것이 자동화의 핵심이다.

4-4. Issue #3, #4 동일 패턴

같은 흐름으로 진행. 빌드 명령이 90% 같은 양식

ISSUE #3

키워드 입력 검증

ISSUE #4

입력 화면 라우팅

이전 Task의 명령 양식을 **90% 복사**해서 진행한다.

자동화 흐름은 모든 Task에 동일하게 적용된다. 패턴을 인식하면, 반복 작업의 시간을 크게 줄일 수 있다.

4-5. 화면 1 동작 3요소 점검

화면 1의 수직 슬라이스 완성

1. 페이지 뜸	입력 폼 표시	완료
2. 인터랙션	사진 업로드 + 키워드 입력	완료
3. 반응	생성 버튼 누름 시 콘솔 출력	완료

각 단계 완료 확인이 화면 1 동작 보증.

Issue #2, #3, #4 모두 통과 시 화면 1 정상 동작이 보장된다.

Step 4 검증 기준

다음 4개 항목 모두 통과 시 Step 5 (화면 2 빌드)로 진입

☐ Issue #2, #3, #4가 모두 PR 병합됐는가

☐ 각 Task가 자동화 흐름으로 진행됐는가

☐ 화면 1의 동작 3요소가 모두 충족됐는가

☐ 작업자의 손이 자료 5의 Issue #1보다 줄었는가

메타 메시지

자동화를 적절히 적용하면, 개발자가 실제로 손으로 해야 할 일이 대폭 줄어든다. Issue #1(수동 전체 처리)에서 Issue #2~#4(자동화 주요 처리)로 진행하면서 경험한 일의 양의 변화가 바로 그것이다.

— STEP 5

화면 2 빌드 (패턴 복사)

Issue #5-#10을 패턴 복사로 빌드해 화면 2 완성 — 자료 6의 확장에 해당

- 01 Issue #5 브랜치 생성
- 02 Issue #5 빌드 명령 (패턴 복사)
- 03 Issue #6-#10 동일 흐름
- 04 각 Task 자동화 (PostToolUse + Stop)
- 05 화면 2 완성 여부 점검

5-1. Issue #5 브랜치

화면 2의 첫 Task. main 동기화 후 feat 브랜치 생성.

```
git checkout main && git pull origin main
git checkout -b feat/task-2-1-openai-client
```

브랜치명 패턴: feat/task-{화면번호}-{설명}. 본 sprint는 화면 2의 첫 Task = task-2-1.

5-2. Issue #5 빌드 명령 (패턴 복사)

화면 1 명령 양식 그대로. 화면 목적과 파일명만 변경.

"Issue #5를 진행해줘.

CLAUDE.md와 rules/architecture.md를 참조해.

PRD의 카피 생성 기능을 위한

OpenAI API 클라이언트를 만들어줘.

기능은 다음 두 가지:

1. GPT-4o 모델 호출을 위한 클라이언트 초기화
2. 환경 변수로 API 키를 읽어 클라이언트 인스턴스 export

이 PR은 src/lib/openai-client.ts 한 파일만 수정해."

패턴: "PRD의 [화면명]을 만들어줘 / 기능은 두 가지 / 이 PR은 N파일만" 양식 재사용. 작업자는 화면 목적과 파일명만 변경.

5-3. Issue #6-#10 동일 흐름

같은 패턴으로 진행. 각 Task는 PostToolUse + Stop 자동화 적용.

ISSUE	작업
Issue #6	프롬프트 템플릿
Issue #7	카피 생성 API
Issue #8	결과 컴포넌트
Issue #9	복사 버튼
Issue #10	결과 라우팅

각 Task 자동화: PostToolUse 후 prd-reviewer 보고서 / Stop 후 git-committer 자동 커밋.

5-4. 화면 2 동작 3요소 점검

화면 2의 수직 슬라이스 완성

1. 페이지 뜸	결과 화면 표시	완료
2. 인터랙션	화면 1에서 화면 2로 라우팅	완료
3. 반응	카피 3개 표시, 복사 버튼 작동	완료

각 단계 완료 확인이 화면 2 동작 보증.

Issue #5-#10 모두 통과 시 화면 2 정상 동작이 보장된다.

5-5. 자동화 효과 측정

자료 5의 Issue #1 (수동) 대비 자료 6의 Issue #5 (자동화) 차이

항목	ISSUE #1 (수동)	ISSUE #5 (자동화)
빌드 명령 입력	새로 작성	메모 복사
PRD 확인	손으로	hook이 자동
커밋 작성	손으로	hook이 자동
작업 시간	100%	50% 이하

작업자의 손이 절반 이하로 줄었다. 자동화의 효과는 측정 가능한 시간 절감으로 드러난다.

— STEP 5 · 검증 기준

다음 Step 진입 가능한가

4개 항목 모두 통과 시 Step N (정리와 비용 비교)로 진입.

☐ Issue #5-#10이 모두 PR 병합됐는가

☐ 화면 2의 동작 3요소가 모두 충족됐는가

☐ 화면 1에서 화면 2로의 라우팅이 자연스러운가

☐ 화면 2 빌드 시간이 화면 1보다 짧아졌는가

— STEP N

정리와 비용 비교

본 단계 목표는 산출물 확인과 토큰 비교, 회고.

- 01 산출물 7개 정리 및 다음 단계 활용
- 02 자동화 토큰 비용 비교 분석
- 03 회고 3개 질문으로 자동화 본질 인식

N-1. 산출물 정리

7개 산출물의 다음 단계 활용처

산출물	다음 단계 활용
동작하는 화면 2개	E2E 테스트 대상
prd-reviewer	다음 단계 검증 도구
git-committer	다음 단계 커밋 도구
PostToolUse + Stop hook	자동화 인프라
자동 검증 보고서 누적	패턴 분석 가능
자동 커밋 히스토리 9+	진행 추적
병합된 PR 9개	작업 완료 증거

— 활동 N-2 · 비용 비교

N-2. /cost 비교

본 sprint 자동화 도구 개발 비용 (토큰)을 확인

/cost

Sub-agent의 컨텍스트 격리 덕분에, 같은 tool을 여러 단계에서 호출해도 각 단계의 토큰 비용이 누적되지 않는다. 이것이 자동화 설계의 핵심 이득이다.

N-3. 회고: 좋은 답 vs 나쁜 답

자동화의 본질을 인식하기 위한 회고 기준

질문	좋은 답	나쁜 답
Issue #1 vs Issue #2 시간 차이	"절반 줄어듦. 명령 복사 + hook 자동"	"빨라졌다"
PRD 검증 보고 불일치	"Issue #7에서 영어 카피 임의 추가 발견. 거부"	"별 거 없었다"
Stop hook 자동 커밋 효과	"커밋 메시지 작성 시간 0. 형식도 자동 일관"	"편했다"

— 활동 N · 검증

Step N 검증 기준

다음 4개 항목 모두 통과 시 학습 단계 완료

☐ 산출물 7개가 모두 손에 있는가

☐ /cost 비교를 메모했는가

☐ 회고 3개 질문에 모두 답했는가

☐ 다음 학습 단계 진입 준비가 됐는가

메타 메시지: Step 0~Step 5까지 5주간 자동화의 본질을 경험했다. 그 경험은 이후 학생의 모든 프로젝트에 적용될 원칙이다.

과제

본 자료의 과제는 [lectures/S6/assignment.md](#) 파일에 정리되어 있다

항목	내용
목표	화면 1:2 완성과 자동화 도구 안정화
산출물	두 화면 동작, sub-agent 2개, hook 2개, Issue #2-#10 PR 병합
마감	다음 학습 단계 시작 전

화면 두 개가 모두 동작 3요소 충족하고 자동화 도구가 안정 작동하는 상태가 다음 단계 출발선.

— TIPS 1

확장과 자동화

- 01 두 단어 분리 — 확장(패턴 복사) vs 자동화(도구 위임)
- 02 필요성에서 출발 — 9개 미작업 Task의 반복 압박

— TIPS 1 · 핵심 개념

두 단어 분리 + 필요성에서 출발

확장 vs 자동화

확장 = 패턴 복사: 화면 1에서 사용한 작업 패턴을 화면 2에 90% 재사용.

자동화 = 도구 위임: 반복 작업을 sub-agent와 hook에 맡김.

복사 대상 예시:

- ① 빌드 명령 메모 (prompt 양식)
- ② Issue 분할 + PR 흐름
- ③ prd-reviewer / git-committer 호출 패턴
- ④ 커밋 메시지 형식

필요성에서 출발

자료 5의 9개 미작업 Task가 **반복의 압박**이 본 단계의 출발점이다. 도구를 먼저 알고 적용처를 찾는 순서는 효과가 떨어진다.

— TIPS 2

Agent 설계

- 01 6원칙 모두 적용 — 한 원칙 무너지면 위험
- 02 tools 최소 권한 — 코드 리뷰어가 코드 수정 못 하게

— TIPS 2 · 핵심 개념

6원칙 + tools 최소 권한

6원칙 모두 적용

단일 책임, 최소 권한, 결정적 출력, 책임 분리, 호출 시점 명시, 입출력 계약. 한 원칙이 무너지면 agent 설계 전체가 위험해진다.

tools 최소 권한

코드 리뷰어는 Read, Grep, Glob만 가능하게. 커밋 sub-agent는 Bash와 Read만. 검증과 자동화의 권한이 섞이면 실패하기 쉽다.

— TIPS 3

hooks

- 01 두 핵심 패턴 — PostToolUse 검증 + Stop 커밋
- 02 CLAUDE.md vs hook — 70% 권고 vs 100% 강제
- 03 matcher 정규식 주의 — 공백 X

— TIPS 3 · 핵심 팁

hooks 핵심 팁 3가지

두 핵심 패턴

PostToolUse에서 자동 검증, Stop에서 자동 커밋. 이 두 패턴을 결합하면 한 Task가 명령한 줄로 끝난다.

CLAUDE.md vs hook

CLAUDE.md는 70% 권고, hook은 100% 강제. 반드시 일어나야 하는 검증과 커밋은 hook으로 자동화한다.

정규식 주의

matcher 정규식에 공백을 넣으면 매칭이 실패한다. Edit와 Write 사이에 공백을 넣지 말 것.

— TIPS 4

검증 우선주의

01 sub-agent 보고도 AI 산출물 — 본인이 직접 확인

— TIPS 4 · 핵심

sub-agent 보고도

AI 산출물

본인이 직접 확인하자

PRD와 일치라고 적힌 항목도 본인이 PRD를 직접 본다. 자동 커밋된 메시지도 git log에서 확인한다. 자동화는 도구일 뿐, 책임은 여전히 사용자에게 있다.

— TIPS 5

빌드 효율

01 시간 절반 = 자동화 작동 — 화면 2가 화면 1의 절반 이하

— TIPS 5 · 마무리

시간 절반 = 자동화 작동

화면 2가 화면 1의 **절반 이하**면 성공

빌드 명령 메모 90% 재사용 + hook 자동 처리. 두 번째 화면이 첫 번째 화면 시간의 절반 이하면 자동화가 제대로 작동하는 것이다.